

Basic Abstractions in Distributed Algorithms

Based on: Cachin, Guerraoui, Rodrigues

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad
Applied algorithms in Control Systems

March 2026.

Framing the Model

Challenges of Distributed Systems

- **Real systems are inherently complex**
 - Many machines are interacting over a network.
 - Concurrent and parallel events can cause unpredictable execution order.
- **Networks are unreliable**
 - Messages may be delayed, lost, or duplicated.
 - Delivery and its order are not guaranteed.
- **Processes may fail**
 - Machines can crash unexpectedly.
 - Worse than that they may behave incorrectly or maliciously.

Key Challenge

Distributed algorithms must operate correctly despite all uncertainties above.

Why We Need Abstractions

- **Distributed systems require formal guarantees**
 - Safety must not depend on timing assumptions.
 - Algorithms must work under uncertainty.
- **Abstractions simplify complex systems**
 - They isolate essential properties of communication.
 - They hide unnecessary implementation details.
- **Abstractions enable formal reasoning**
 - Algorithms can be analyzed mathematically.
 - Because of that properties such as reliability and agreement become provable.

Key Idea

Chapter 2 from the book introduces the fundamental building blocks of distributed systems. Instead of implementing full systems immediately, we first want to understand abstractions that will allow us to reason about correctness and guarantees.

- **Single-threaded execution**

- The simulator executes one step at a time.
- This makes system behavior much easier to analyze.

- **No goroutines, no networking**

- We avoid real concurrency and definitely real networks.
- The goal for now is to model distributed behavior, not infrastructure.

- **Explicit nondeterminism**

- Message delivery order is chosen randomly.
- This will model asynchronous distributed execution without overcomplicating things at the start.

Process Model

State Machine Model

State Machine

A **state machine** is a computational model defined by:

- a set of **states** describing the internal configuration
- a set of **events or inputs**
- a **transition function** that determines how the state changes

Formally, the transition function is:

$$(state, input) \rightarrow (new\ state, outputs)$$

Distributed Process

In distributed algorithms, **process** is modeled as a state machine:

$$(current\ state, message) \rightarrow (new\ state, outgoing\ messages)$$

Process Interface

Process Structure

- `ID()` - returns the process identifier
- `Handle(msg)` - processes an incoming message

Pseudo Interface

```
type Process interface {  
    ID() string  
    Handle(msg Message) []Message  
}
```

Important

A **thread** is an operating system execution unit.

A **process** in the distributed model is an **abstract state machine** that can be implemented by a thread, goroutine, or a machine in a real system.

Message Abstraction

- Processes communicate by sending **messages**.
- A message contains:
 - sender identifier
 - destination identifier
 - message type
 - optional payload data

Message Structure

```
type Message struct {  
    From ProcessID  
    To    ProcessID  
    Type  string  
    Data  any  
    Meta  map[string]any  
}
```

All Communication Is Mediated

- Processes **never** call each other directly.
- Message delivery is controlled by the **runtime system**.
- The scheduler decides **when and which message is delivered**.



Asynchronous Execution Model

- There is **no global clock**.
- Processes run at **arbitrary speeds**.
- Message delays are **unbounded**.
- Message delivery is not guaranteed unless the **communication link** says so.

Key Idea

The system makes **no timing assumptions**. Correctness must not depend on message delays or process speed.

Nondeterministic Execution

- Multiple events may be possible at the same time.
- The runtime (scheduler) chooses which message is delivered next.
- Different choices produce **different executions**.

Key Consequence

The same program can produce **different execution traces**.

Example:

$send(m_1), send(m_2)$

Possible deliveries:

$m_1 \rightarrow m_2$ or $m_2 \rightarrow m_1$

Execution Trace

- It can be seen how distributed system works through a sequence of **events**.
- Examples of events:
 - message send
 - message delivery
 - process state transition
- Different message orders produce different **executions**.

Formal Definition

An **execution trace** is an **ordered sequence of events**

$$e_1, e_2, e_3, \dots$$

that describes how the state of a distributed system evolves during an execution.

Example Execution Trace

Two processes exchanging messages:

- Step 1: P1 sends PING to P2
- Step 2: message delivered to P2
- Step 3: P2 processes PING
- Step 4: P2 sends PONG
- Step 5: message delivered to P1
- Step 6: P1 processes PONG

Key Idea

Distributed algorithms are analyzed by studying their possible execution traces.

Example: Ping-Pong

- Two processes: **P1** and **P2**
- Communication is done through a **perfect link**
- The processes exchange **PING** and **PONG** messages
- The protocol runs for **3 message exchanges**

Purpose of the Example

It demonstrates the properties of a **Perfect Link**:

- PL1: reliable delivery
- PL2: no duplication
- PL3: no creation

Link Hierarchy

Why Abstract Links?

- Real networks are **unreliable**.
- Different systems require different **guarantees**.
- We build stronger guarantees **by composition**.

Key Idea

Link abstractions let us reason about communication **independently** from the physical network.

Fair-Loss Link (FLL)

- Messages **may be lost**.
- **Finite** duplication is allowed.
- **No creation**: messages are not invented.
- Delivery is **probabilistic**.

Interpretation

This is the **weakest** link abstraction.

Stubborn Link (Algo 2.1)

- Built on top of **Fair-Loss Link**.
- Keeps a **retransmission buffer**.
- Retransmits stored messages **forever**.
- If delivery is possible, a sent message is delivered **infinitely often**.

Composition Formula

$$\text{Stubborn} = \text{FairLoss} + \text{Retransmission}$$

Perfect Link (Algo 2.2)

- **Reliable delivery**: every sent message is delivered.
- **No duplication**: delivered at most once.
- **No creation**: only sent messages can be delivered.
- Built using:
 - **Stubborn** link
 - **Deduplication set** (delivered IDs)

Composition Formula

$$\text{Perfect} = \text{Stubborn} + \text{Dedup}$$

Authenticated Perfect Link

- Ensures **authenticity**: sender cannot be forged.
- Attaches a **MAC tag** on send.
- Verifies the MAC on receive.
- Forged or modified messages are **rejected**.

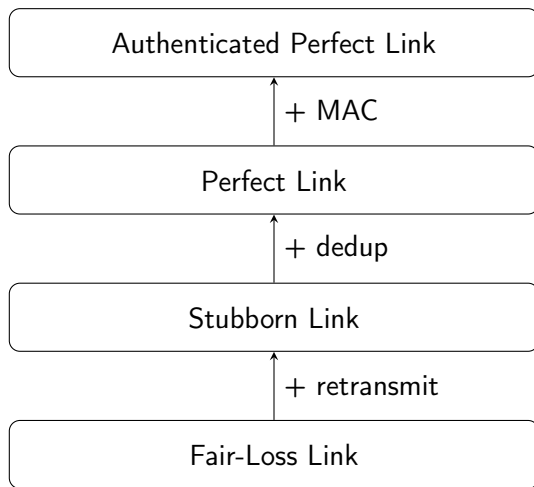
Composition Formula

Authenticated = Perfect + MAC

MAC Mechanism

```
send:    tag = HMAC(key, message)
recv:    accept only if HMAC(key, message) == tag
```

Hierarchy of Link Abstractions



Failure Models

Failure Models Comparison

Model	What can go wrong?
No Failure	All processes behave correctly
Crash-Stop	Process halts permanently
Omission	Process may drop messages
Byzantine	Process can behave arbitrarily

Key Point

Each stronger model breaks **more assumptions** about process behavior.

Crash-Stop Model

- A process **halts** during execution.
- It never recovers (**no restart**).
- After crashing, it sends **no more messages**.

Intuition

A crashed process behaves like it was suddenly **turned off**.

Omission Failure

- A process is still **alive**.
- It may **drop messages** (send or receive omissions).
- The failure is **silent**, which means no explicit error is reported.

Key Difference

Crash-stop means **no steps at all**. Omission means **steps continue**, but communication may fail.

Byzantine Model

- A faulty process can behave **arbitrarily**.
- It may send **wrong values**.
- It may **forg**e messages or alter contents.
- It may **impersonate** other processes.

Key Point

This is the **strongest and hardest** failure model.

Why Failure Models Matter

- Distributed algorithms must tolerate **faults**.
- The **failure model** defines what kinds of faults the algorithm must handle.
- Stronger failure models require **more complex algorithms**.

Key Insight

The choice of a failure model determines **what correctness means** for a distributed algorithm.

Cryptographic Primitives

Why is Cryptography needed in Distributed Systems?

- **Integrity:** detect if a message was modified.
- **Authentication:** verify who sent the message.
- **Non-repudiation:** prove that a sender created a message.

Motivation

In untrusted environments, processes may try to **tamper with or forge messages**.

Hash Function (SHA-256)

- Produces a **fixed-length digest**.
- Any change in the input produces a **different hash**.
- Used to verify **data integrity**.

Example

`hash(message) → digest`

If the message changes, the digest also changes.

Message Authentication Codes (HMAC-SHA256)

- Uses a **shared secret key**.
- Sender computes a **MAC tag**.
- Receiver verifies the tag using the same key.

Basic Mechanism

$$\text{tag} = \text{HMAC}(\text{key}, \text{message})$$

If verification fails → the message is **rejected**.

MAC Verification Example

Honest Message

```
message = "TRANSFER 100"  
tag = HMAC(key, message)  
  
send(message, tag)
```

Receiver Verification

```
if HMAC(key, message) == tag {  
    accept(message)  
} else {  
    reject(message)  
}
```

Tampering Attempt

Attacker modifies message:

"TRANSFER 100"

→

"TRANSFER 1000"

Receiver recomputes MAC.

Verification fails → message rejected.

Safety vs Liveness

Safety Property

- *Bad things never happen*
- Violations can be detected by observing a **single execution step**

Liveness Property

- *Good things eventually happen*
- Cannot be falsified by **finite execution**

Key Idea

Safety violations appear in a **finite prefix of a trace**. Liveness violations require **infinite observation**.

Counter over Fair-Loss

- A process sends **10 increment** messages.
- Messages are delivered through a **Fair-Loss Link**.
- Some messages may be **lost**.

Observation

- Counter value **never exceeds 10** → **Safety holds**
- Some increments may never arrive → **Liveness may fail**

Conclusion

Safety $\neq$ Liveness

Runtime Scheduler

Execution of One Scheduler Step

- ① Perform pending **retransmissions**
- ② Select a **random message** from the network
- ③ Apply the **failure model**
- ④ Apply **link abstraction logic**
- ⑤ Deliver the message to the **destination process**
- ⑥ Process updates its state and produces **outgoing messages**
- ⑦ Send outgoing messages through the **link abstraction**
- ⑧ Record the event in the **execution trace**
- ⑨ Check for **safety violations**

Key Idea

The scheduler **just simulates** asynchronous distributed execution.

Big Picture

What We Achieved

- A **formal model of processes** as state machines
- A hierarchy of **communication link abstractions**
- Explicit **failure models**
- **Cryptographic authenticity** for communication
- Runtime detection of **safety and liveness properties**

Outcome

We built a **simulator** for distributed system executions.

- These abstractions form the basis for many distributed algorithms.

Next Topics

- Reliable Broadcast
- Consensus
- Blockchain protocols

Key Insight

Almost every distributed algorithm is built on top of these abstractions.